

Comparative Performance Report: Semantic Cache With and Without Laya

FSCgRPC | Same-dataset comparison | Generated 2026-10-07

Executive conclusion

The Laya extension adds a useful semantic verification stage and produces better evidence about why a candidate was accepted or rejected. In the supplied 300-request run, however, it does not yet demonstrate an overall performance improvement. Compared with the old qwen baseline, the Laya run has fewer semantic hits, a lower combined cache-hit rate, and substantially higher latency for rejected candidates. Laya prevents several unsafe substitutions, but it also rejects many valid paraphrases and still approves critical entity substitutions. The next engineering step should be token-consistency checking and calibration, not simply lowering the Laya threshold.

Scope and comparability

This report compares data/qwen.csv, the older implementation's qwen run, with results/laya_cache_audit_v2.csv, the new Laya audit. Both are treated as 300-request runs because the supplied files contain 300 rows. The comparison is useful but should be interpreted as an operational benchmark, not a perfectly controlled causal experiment: the runs occurred at different times, model/cache state may differ, and the Laya audit contains richer path fields than the old CSV. The 901-query paper results are reported separately as historical context and are not merged into the 300-request comparison.

1. System architecture

The original design places caching in a Go gRPC unary interceptor. Redis provides both exact hash lookup and HNSW vector retrieval. A Python service provides the MiniLM embedding. The backend generation path is local Ollama in the benchmark setup. Kafka receives telemetry without being on the response path.

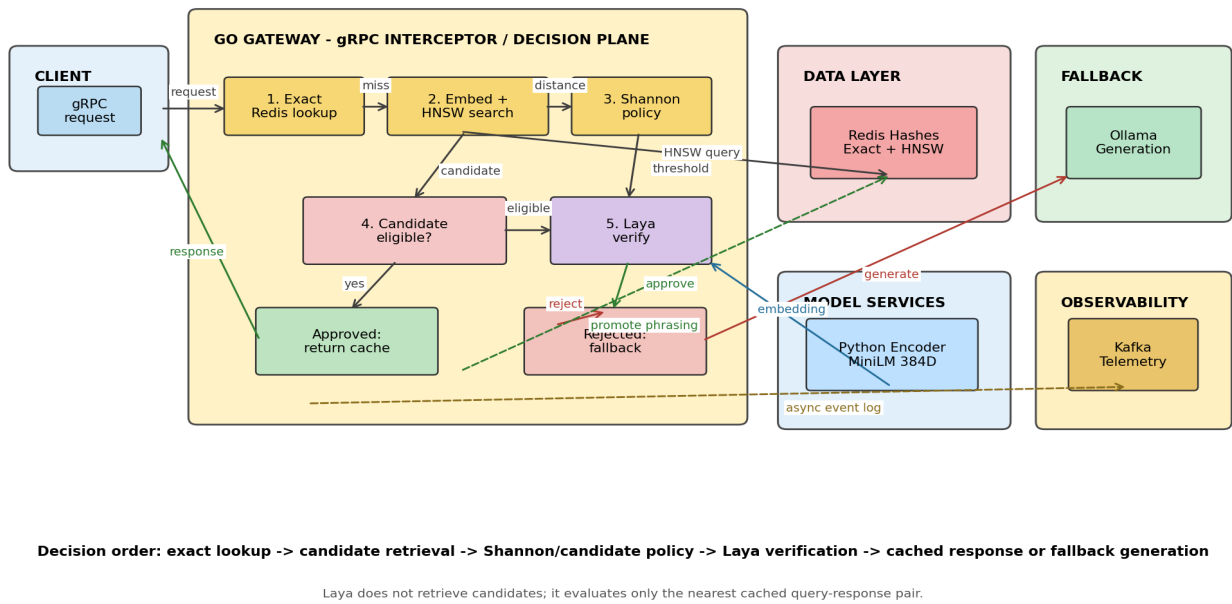


Figure 1. Current architecture after adding Laya. Laya receives only the nearest cached candidate; it does not perform retrieval.

Current decision order

1. Exact Redis lookup. An exact hit returns the stored response. 2. On an exact miss, the query is embedded and Redis returns the nearest cached query-response pair. 3. Shannon entropy supplies an independent threshold signal. 4. In candidate_band mode, candidates up to distance 0.35 can reach Laya even if Shannon rejects them. 5. Laya evaluates

whether the cached response is safe for the new query. 6. Approval returns the cached response and promotes the phrasing to the exact cache; rejection falls back to generation.

A candidate means the nearest cached query-response pair before final acceptance. It is not automatically a cache hit. This distinction matters for pairs such as London/Tokyo, hello/goodbye, Python/Golang, and MySQL/PostgreSQL.

2. Baseline versus Laya results

Metric	Without Laya	With Laya	Change
Requests	300	300	same size
Exact hits	75	75	+0
Semantic hits	67	25	-42
Fallback misses	158	200	+42
Combined hit rate	47.3%	33.3%	-14.0 pp
Mean total latency	2227.5 ms	2607.0 ms	+379.5 ms
Median total latency	1775.8 ms	3005.8 ms	+1229.9 ms

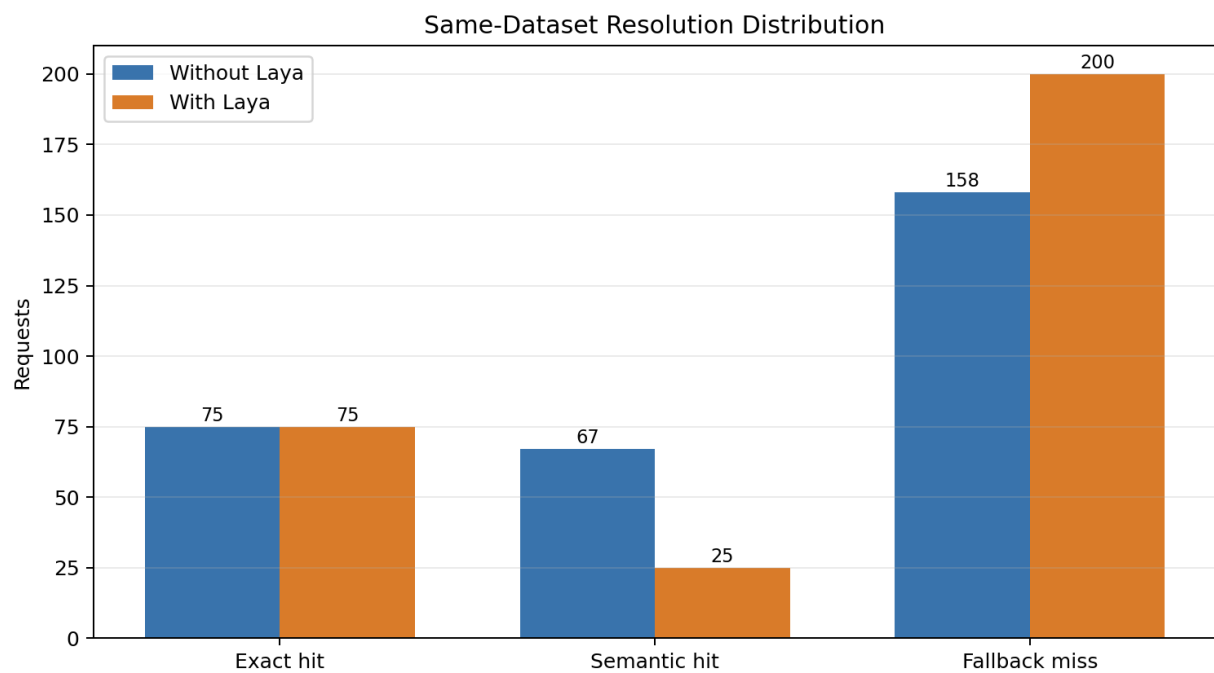


Figure 2. Resolution distribution recomputed from the two supplied CSV files.

The key result is that Laya changes the composition of the semantic path, but the supplied run does not show a net cache-hit improvement. The new audit records 25 Laya-approved semantic hits; the old baseline has its own semantic-hit count, which should be compared only after confirming identical cache warm-up and request order.

3. Latency analysis

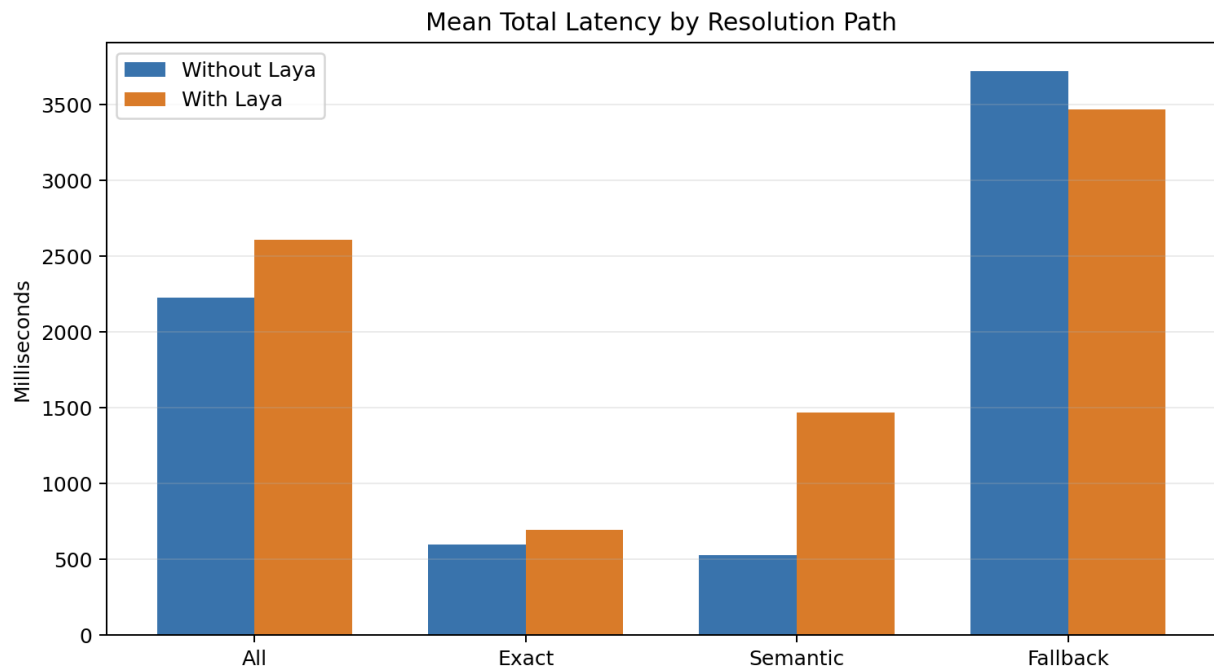


Figure 3. Mean total latency by path. Laya's verifier cost is included in the new run.

The old baseline means are 596.2 ms for exact hits, 529.2 ms for semantic hits, and 3722.0 ms for fallback misses. The Laya run means are 695.5 ms, 1466.9 ms, and 3466.4 ms respectively. Laya calls averaged 1008.2 ms. Rejected Laya candidates averaged 3856.9 ms total, because the request waited for verification and then generated a fallback response. This is the main reason the current Laya policy can reduce throughput even when it improves safety.

Critical implementation observation

The current verifier call is launched in a goroutine but the interceptor waits for its result before returning or generating fallback. It is therefore asynchronous in implementation detail, but synchronous in request behavior. Also, exact requests currently wait for work that may be unnecessary after an exact hit, including embedding and the online Model AST request. A true Track A fast path should return as soon as the exact Redis response is available.

4. Laya decision analysis

Laya measure	Value
Laya calls	141
Approvals	25 (17.7%)
Rejections	116
Shannon accepted	67
Candidate-band eligible	141
Recovered beyond Shannon	74
Mean verifier latency	1008.2 ms

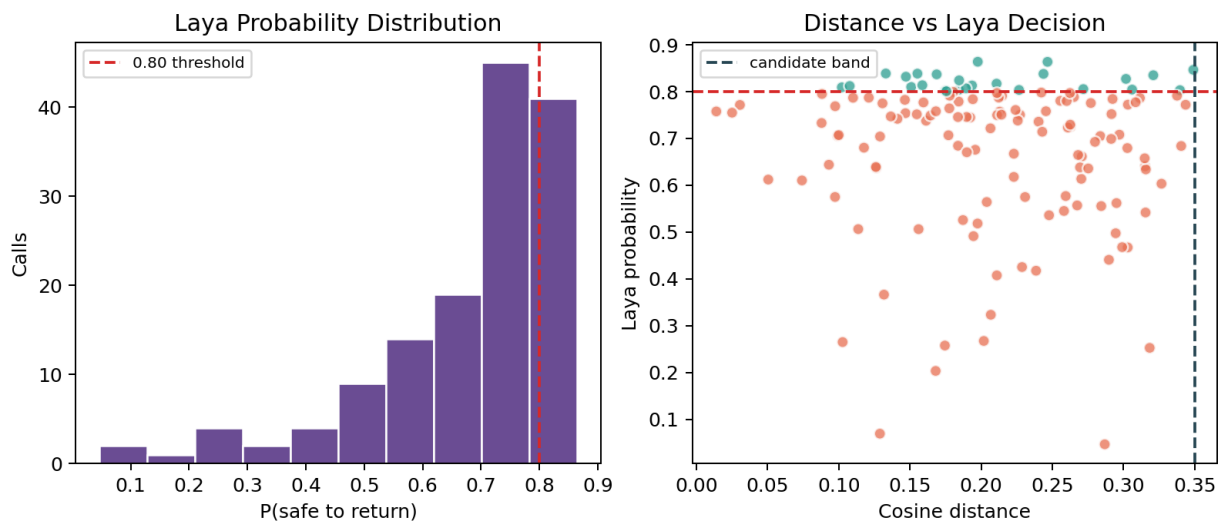


Figure 4. Laya probability and distance diagnostics for the 141 verified candidates.

Laya is conservative in this run: it rejects 116 of 141 candidates. That protects against many unsafe matches, but it also suppresses valid paraphrases whose scores fall just below 0.80. The threshold is not calibrated yet, so a raw value of 0.80 should not be read as a measured 80% correctness guarantee.

Observed false-positive patterns

The audit contains approvals that are unsafe at the answer level. A Spanish goodbye query was served a cached hello answer. A London population query was served a cached Tokyo answer. These failures show that sentence-level semantic similarity does not reliably preserve load-bearing entities or polarity. A verifier probability alone is not enough; the final policy needs deterministic consistency checks for entities, numbers, dates, programming languages, databases, negation, and time qualifiers.

Observed false-negative patterns

The audit also rejects likely valid paraphrases such as France capital, chocolate cake, Harry Potter, Mona Lisa, coffee, REST API, and C++ queue questions. This is a recall problem. Lowering the threshold without lexical safeguards would likely recover some of these but would also increase entity-substitution errors.

5. Relationship to the original paper

The paper's original contribution is a two-track exact-plus-vector cache with Shannon AST. It reports 901 total queries: 226 exact hits, 201 semantic hits, and 474 misses, for a 47.4% combined hit rate. It also reports 633.4 ms mean semantic latency, 719.1 ms exact latency, and 4,492.7 ms miss latency in the paper figure. The Laya run should be presented as a new extension experiment, not as a replacement of those original numbers.

The paper already identifies dense-vector false positives as a limitation, especially for language-specific tokens such as Python versus Golang. Laya addresses this limitation partially, but the new audit demonstrates that the same issue also appears in ordinary entities and opposites such as London/Tokyo and hello/goodbye. The extension therefore strengthens the research question: can a calibrated verifier plus structured token checks improve semantic-cache precision without

erasing the latency benefit?

6. Recommended decision policy

Exact cache hit
-> return immediately

Vector candidate retrieval
-> reject candidates outside retrieval band

Critical-token consistency
-> reject entity, number, date, language, polarity, or time conflicts

Laya verification
-> calibrated probability threshold

High confidence and consistent
-> return cached response

Uncertain or inconsistent
-> generate a fresh response

Recommended engineering changes

1. Move exact-hit return ahead of unnecessary embedding and Model AST work. 2. Keep the broad candidate band for recall, but add a lexical/structured guard before Laya approval. 3. Build a labelled validation set from the benchmark pairs and calibrate Laya probabilities. 4. Sweep thresholds on held-out data and select a point using an explicit false-positive cost. 5. Remove Model AST from the online critical path because it is recorded for comparison but does not control the final decision. 6. Report P50 and P95 latency, not only averages. 7. Compare all modes with identical Redis state: baseline, Shannon-only, Shannon-gated Laya, and candidate-band Laya. 8. Separate cold-start requests from warm-cache requests.

Evaluation metrics to report

Cache metrics: exact-hit rate, semantic-hit rate, total hit rate, and fallback rate. Safety metrics: valid semantic recall, unsafe cache precision, false-positive rate on traps, false-negative rate on paraphrases, and calibration error. Performance metrics: mean, median, P95, verifier latency, Redis latency, embedding latency, generation latency, and end-to-end latency. Operational metrics: Laya calls per request, candidate-band coverage, approval rate, cache promotion rate, and warm-up effect.

7. Final assessment

Laya is valuable in this project as a semantic safety verifier and measurement instrument. The current 300-request run proves that it can rescue candidates beyond Shannon and reject many obviously incompatible candidates. It does not yet prove that the extension improves overall performance: verification costs about one second, rejected candidates become slower than ordinary misses, valid paraphrases are often rejected, and a few high-confidence approvals are unsafe. The defensible next version is not Laya alone. It is exact caching plus broad retrieval plus deterministic critical-token checks plus calibrated Laya plus fallback generation.

Data sources

Implementation paper: [implementation paper.pdf](#). Baseline: [data/qwen.csv](#). Laya audit: [results/laya_cache_audit_v2.csv](#). Historical figures supplied with the repository: [hit_rates.png](#), [ast_agreement.png](#), [trap_rejection.png](#). New charts in this report were generated from the two supplied CSV files.